



Nintendo tops them all

According to ESA Nintendo Switch topped the competition in unit sales and dollars in 2018 followed by PS4. <https://dgit.in/feb19-43>



Opera Touch on iPad

Opera has announced Opera touch for iPad and a 'new' private mode on its Android and iPhone versions. <https://dgit.in/feb19-44>

scope for any runtime anomalies. This leads to faster execution of programmes that use AOT. Dart, being able to use both, invariably puts itself in an advantageous position of being used for Flutter. Quickened development and faster startup is what we get from using Dart, thereby leading to a very expeditious app development.

One of the biggest highlights of Dart would definitely have to be the 'Stateful Hot Reload'. Press a key-combo only to see your entire code being compiled, modified (if any) and refreshed with the changes in place, all of it up and running in a time of milliseconds.

```

E/eglCodecCommon( 5856): glUtilsParamSize: unknow param 0x00000741
E/eglCodecCommon( 5856): glUtilsParamSize: unknow param 0x00000741
I/flutter ( 5856): hi there
I/flutter ( 5856): hi there

Initializing hot reload...
Reloaded 0 of 420 libraries in 453ms.

```

Flutter is blazing fast

We love this for its trailblazing speed! This has been and will be a boon for all the developers out there who have had to patiently stand by the clock to see their code be deployed. Not anymore though, with this state-of-the-art groundbreaking speed that the hot reload brings. The state of the app is stored at every reload and can be picked up from where it had been left.

Using Dart, we can create animations that run with smooth finesse at 60 frames per second, thereby ensuring the best user and developer experience. Dart also manages to add to its arsenal the mechanism of object allocation and garbage collection without the use of locks. Fascinating, isn't it! If you want to know more, read about it here: <https://dgit.in/Flutter>.

To put in brief, Flutter uses threads called isolates that are not run preemptively, thereby not allowing the sharing of memory leading to no such situation where problems such as deadlocks might occur.

DIY

It's time we get into what Flutter can do and probably try it for ourselves. We begin with importing of pack-

ages. Yes, C++, Java or any other object oriented programmers, you've started feel comfortable, haven't you? Don't worry. We aren't not going to take you out of your zone either. You'll be able to breeze through it. And for others, all this needs is a bit of focus.

Think of importing packages to be adding a few necessary files to your code that you shall have to use more often. In our case, we shall import a class called `material.dart` among other classes that comes with the default flutter package. As usual, we can also import classes through packages which have been written by us as

well. Material is a visual design system which can be coded in open source and off late has become a standard for mobile and web applications widely accepted and lauded by developers. Next up, we need to define create our main function which serves as the starting point for the execution of the app. It is also the place where we define or call several other methods and/or widgets. We do that in a file called `main.dart` which gets built whenever we create an application from the menu and it resides inside the `lib` folder.

We create another file with any name that we prefer, say, `myapp.dart`. After we are done with this class, we shall call it from the `main.dart`. Since we shall use the material widget, we go ahead and import that. We define the class as `FirstApp` and we extend the `StatelessWidget` to keep things simpler since our app isn't going to keep transitioning between states. Go ahead and type some more, and at this point, let us understand that we shall return a new object of the Material class, i.e., the object which shall be our app itself. Let's use some properties, which you can think of to be variables, to give FirstApp some identity.

Let us understand the code over at <https://dgit.in/Flutter1>. This bit of code essentially describes the part of the UI that the widget represents. What this does is replace the subtree of widgets (widgets, parents and their children with furthermore nesting results in a tree like structure) by updating the existing nested widget tree or by removing and placing itself as the new subtree provided the widget returned can update the root of the existing tree. Through this method the existing context of the widget is passed as a object of the `BuildContext` Class and after being modified it's returned at its correct position in the widget tree.

We can set the color like this.

```
color: Colors.red;
```

The body consists only of a simple text defined inside a text widget.

We can also define a child (or children in cases of a list) to any widget. This feature allows nesting (similar to the way we looped inside loops) and we can go into any amount of detail until we are satisfied. For every child, we instantiate a new widget which necessitates the usage of the new keyword. Let us add a new inkwell widget as a child to the Material widget in which we are in right now. An inkwell widget responds to a touch in a rectangular area on the screen.

```
child: new InkWell(...)
```

Let's nest another widget here called the column widget which can be used to list multiple objects. As you might have guessed it's going to have children in here and likewise the syntax is `children`. While setting a children property, we need to place the Widget class in angular brackets much like our generic classes in java. We can add as many records/rows to this column, each record being capable of holding any kind of widget. Let's add two text widgets. Dart allows us to control styling, colors, alignment, sizes – almost anything that you can think of. At any given point of time you might as well feel that you're developing an HTML or a CSS file. Check out the code at <https://dgit.in/Flutter2>.